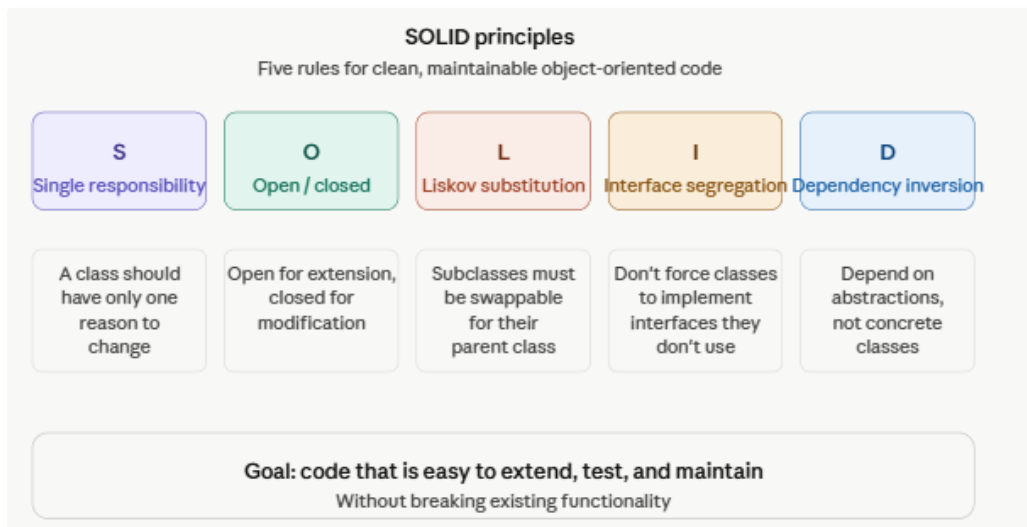


SOLID principles



S — Single Responsibility Principle

A class should have only one reason to change — one job, one responsibility.

When a class handles too many things (logic, formatting, persistence), changes to one area break others. Split them apart so each class owns one clear concern.

O — Open / Closed Principle

Classes should be open for extension but closed for modification.

Instead of editing existing code every time you add new behaviour (risking bugs), design classes so you extend them via new subclasses or strategy objects.

```
1 class DiscountCalculator {
2     public double calculate(String type, double price) {
3         // Adding a new discount type = editing THIS class ← problem!
4         if (type.equals("student")) return price * 0.8;
5         if (type.equals("employee")) return price * 0.7;
6         if (type.equals("vip")) return price * 0.6;
7         return price;
8     }
9 }
10
11 //After
12
13 interface DiscountStrategy {
14     double apply(double price);
15 }
16
17 // Each discount is its own class – no touching existing code
18 class StudentDiscount implements DiscountStrategy {
19     public double apply(double p) { return p * 0.8; }
20 }
```

```

21 class EmployeeDiscount implements DiscountStrategy {
22     public double apply(double p) { return p * 0.7; }
23 }
24 class VipDiscount implements DiscountStrategy {
25     public double apply(double p) { return p * 0.6; }
26 }
27
28 class PriceCalculator {
29     public double calculate(DiscountStrategy strategy, double price) {
30         return strategy.apply(price);
31     }
32 }

```

L — Liskov Substitution Principle

- Subclasses must be usable wherever their parent class is expected — without breaking the program.

If you swap a parent for a child and something breaks or throws unexpectedly, you've violated LSP. A child class must honour all the contracts (behaviour, exceptions, return types) of its parent.

```

1 class Bird {
2     public void fly() {
3         System.out.println("Flying!");
4     }
5 }
6
7 // Penguin IS-A bird, but can't fly ← breaks substitution!
8 class Penguin extends Bird {
9     public void fly() {
10         throw new UnsupportedOperationException(
11             "Penguins can't fly!"
12         );
13     }
14 }
15
16 // This call will crash at runtime:
17 Bird b = new Penguin();
18 b.fly(); // 🌟 UnsupportedOperationException
19
20 //after
21
22 interface Bird { String name(); }
23
24 interface FlyingBird extends Bird {
25     void fly();
26 }
27
28 // Only birds that actually fly implement FlyingBird
29 class Sparrow implements FlyingBird {
30     public String name() { return "Sparrow"; }
31     public void fly() { System.out.println("Flying!"); }
32 }
33
34 // Penguin is a Bird, but NOT a FlyingBird — honest!
35 class Penguin implements Bird {
36     public String name() { return "Penguin"; }
37     public void swim() { System.out.println("Swimming!"); }
38 }

```

I — Interface Segregation Principle

Don't force a class to implement methods it doesn't need.

Fat interfaces that bundle unrelated methods force implementors to write dummy or throwing methods. Split large interfaces into smaller, focused ones so each class only implements what it actually uses.

```
1 interface Worker {
2     void work();
3     void eat(); // ← robots don't eat!
4     void sleep(); // ← robots don't sleep!
5 }
6
7 class RobotWorker implements Worker {
8     public void work() { System.out.println("Working"); }
9
10    // Forced to implement meaningless methods:
11    public void eat() { throw new UnsupportedOperationException(); }
12    public void sleep() { throw new UnsupportedOperationException(); }
13 }
14
15 //after
16
17 // Split into focused interfaces
18 interface Workable { void work(); }
19 interface Feedable { void eat(); }
20 interface Sleepable { void sleep(); }
21
22 // Human uses all three
23 class HumanWorker implements Workable, Feedable, Sleepable {
24     public void work() { System.out.println("Working"); }
25     public void eat() { System.out.println("Eating"); }
26     public void sleep() { System.out.println("Sleeping"); }
27 }
28
29 // Robot only implements what it needs
30 class RobotWorker implements Workable {
31     public void work() { System.out.println("Working"); }
32 }
```

D — Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

When a class creates its own dependencies (`new MySQLDatabase()`), it's tightly coupled and hard to test or swap. Instead, inject abstractions so the class doesn't care about the concrete implementation.

```
1 // Low-level: specific database
2 class MySQLDatabase {
3     public void save(String data) {
4         System.out.println("Saving to MySQL: " + data);
5     }
6 }
```

```

5     }
6 }
7
8 // High-level: directly creates the low-level class ← tight coupling!
9 class UserService {
10     private MySQLDatabase db = new MySQLDatabase(); // ← problem
11
12     public void createUser(String name) {
13         db.save(name); // Can't switch to MongoDB, PostgreSQL, etc.
14     }
15 }
16
17 //after
18
19 // Abstraction – neither side knows the other
20 interface Database {
21     void save(String data);
22 }
23
24 // Low-level modules implement the abstraction
25 class MySQLDatabase implements Database {
26     public void save(String d) { System.out.println("MySQL: " + d); }
27 }
28 class MongoDBDatabase implements Database {
29     public void save(String d) { System.out.println("MongoDB: " + d); }
30 }
31
32 // High-level depends on the abstraction, injected from outside
33 class UserService {
34     private final Database db; // ← interface, not a concrete class
35
36     public UserService(Database db) { this.db = db; } // ← injected
37
38     public void createUser(String name) { db.save(name); }
39 }
40
41 // Caller decides which database to use – easy to swap or mock
42 UserService svc = new UserService(new MongoDBDatabase());

```